

SIMATS ENGINEERING

ASSESSMENT TOOL 2

NAME: M.GOKUL
REG : 192565067

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Dr. JAYA MABEL RANI

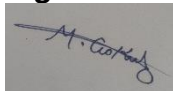
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and	BL4 – Analyze	5

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I M . G O K U L 192565067 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution	25%	Innovative,	Logical and	Basic	Unclear or

Design		practical, and feasible solution aligned with industry standards	feasible solution with minor gaps	solution with limited feasibility	impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. E-commerce – 10 Million Product Records

Question: Recommend a file organization strategy and justify it with cost analysis.

Answer:

For 10 million product records, **Hashed File Organization** is suitable for fast product-ID based retrieval.

- Use ProductID as the hash key.
- Records are distributed into buckets.
- Hashing provides fast direct access.
- Collisions can be handled using chaining.
- It is suitable for large-scale product databases.

Cost Analysis:

Operation Cost

Search	$O(1)$ average
Insert	$O(1)$ average
Delete	$O(1)$ average
Storage	Depends on buckets

Conclusion: Hashed organization provides efficient storage and fast retrieval for 10 million products.

Q2. Banking System – Account Number and Date Range

Question: Design an indexing strategy for fast retrieval by account number and date range.

Answer:

Use two indexes:

1. **B+ Tree index on AccountNumber**
 - Provides fast exact/point searches.
2. **B+ Tree index on TransactionDate**
 - Supports efficient date-range queries.

Example:

AccountNumber $\rightarrow$ B+ Tree Index

TransactionDate $\rightarrow$ B+ Tree Index

For a query such as:

Account = 10025

Date = January 1 to January 31

the indexes can quickly locate the required transactions.

Conclusion: B+ Tree indexes are suitable because they support both equality and range searches efficiently.

Q3. Healthcare – PatientID and Doctor Name

Question: Design a multi-level indexing solution.

Answer:

Use a **multi-level B+ Tree indexing system**.

Primary Index



PatientID B+ Tree



Patient Record

A secondary index is created for doctor name:

Doctor Name Index



Patient Records

- **Primary index:** PatientID.
- **Secondary index:** DoctorName.
- Multi-level indexing reduces disk accesses.
- B+ Tree provides efficient searching.

Conclusion: This design supports fast access using both PatientID and Doctor Name.

Q4. Logistics – B-Tree vs B+ Tree

Question: Analyze whether B-Tree or B+ Tree is more suitable for dynamic shipment insertion and deletion.

Answer:

B+ Tree is more suitable.

Feature	B-Tree	B+ Tree
Search	Fast	Fast
Insert	Efficient	Efficient
Delete	Efficient	Efficient
Range Query	Less efficient	Very efficient
Leaf Linking	No	Yes

B+ Tree stores data records at the leaf level and links the leaves sequentially.

Conclusion: B+ Tree is preferred for logistics because shipment records may require frequent insertion, deletion and range-based retrieval.

Q5. HR System – 5000 Employees

Question: Design a hashing scheme and evaluate static hashing vs extendible hashing.

Answer:

Use **EmployeeID** as the hashing key.

$\text{Hash}(\text{EmployeeID}) = \text{EmployeeID} \% \text{Number_of_Buckets}$

Static Hashing

- Fixed number of buckets.
- Simple implementation.
- Performance decreases when the database grows.
- Overflow buckets may be required.

Extendible Hashing

- Directory can grow dynamically.
- Handles bucket overflow by splitting.
- Directory can double when required.
- Better for changing employee data.

Feature	Static	Extendible
Bucket size	Fixed	Dynamic
Overflow	More likely	Controlled
Growth	Difficult	Easy
Flexibility	Low	High

Conclusion: Extendible hashing is better if the HR database is expected to grow.

Q6. Airline Reservation System

Question: Support point queries by flight number and range queries by date. Propose a combined index structure.

Answer:

Use a **combined indexing strategy**:

FlightNumber → Hash Index

Date → B+ Tree Index

Hash Index

Used for exact flight-number searches.

Example:

FlightNumber = AI202

B+ Tree

Used for date ranges.

Example:

Date BETWEEN 01-08-2026 AND 10-08-2026

Conclusion: Hashing provides fast point queries, while B+ Tree provides efficient date-range queries.

Q7. University Student Database

Question: Design file organization and secondary indexing for fast queries by department and CGPA range.

Answer:

Use **Sorted File Organization** based on StudentID.

Create secondary B+ Tree indexes:

Student File



StudentID

Secondary Index 1 → Department

Secondary Index 2 → CGPA

- Department index supports department-based searching.
- CGPA B+ Tree index supports range queries.
- Example:

CGPA BETWEEN 8.0 AND 9.0

Conclusion: Sorted storage with secondary B+ Tree indexes provides efficient student retrieval.

Q8. Disk Block Utilization

Question: Analyze Heap, Sorted and Hashed file organizations for **100,000 records**, each **50 bytes**, using **4 KB blocks**.

Given:

Records = 100,000

Record size = 50 bytes

Block size = 4 KB = 4096 bytes

Records per block:

$4096 / 50 = 81$ records

Total blocks:

$100000 / 81 \approx 1235$ blocks

Comparison

File Organization Block Usage

Heap Approximately 1235 blocks

Sorted Approximately 1235 blocks

Hashed Approximately 1235+ blocks depending on overflow

Utilization

$81 \times 50 = 4050$ bytes

Unused space:

$4096 - 4050 = 46$ bytes

Utilization:

$4050 / 4096 \times 100 \approx 98.88\%$

Conclusion: Each block can store approximately **81 records**, giving about **98.88% block utilization**.

Q9. Social Media – 500 Million Posts

Question: Design a composite indexing strategy using user_id, timestamp and hashtag.

Answer:

For 500 million posts, use multiple indexes:

(user_id, timestamp) → Composite B+ Tree

hashtag → Secondary Index

Composite Index

(user_id, timestamp)

This supports queries such as:

Find posts of a particular user

during a particular time period.

Hashtag Index

hashtag → Post IDs

This supports hashtag searches.

Conclusion: Composite B+ Tree indexing on (user_id, timestamp) plus a secondary hashtag index provides efficient large-scale post retrieval.

Q10. Supermarket Billing System

Question: Compare insert, delete and search performance of Heap, Sorted and Hashed Files for 1 million daily transactions.

Answer:

Operation	Heap File	Sorted File	Hashed File
Insert	Fast	Slow	Fast
Delete	Slow	Moderate	Fast
Search	Slow	Fast	Very Fast
Range Search	Slow	Fast	Poor

Explanation

Heap File:

- Fast insertion.
- Searching requires scanning records.

Sorted File:

- Fast searching using binary search.
- Insertions may be expensive because records must remain sorted.

Hashed File:

- Very fast exact search.
- Efficient for transaction ID/bill number lookup.

Conclusion: For a supermarket billing system with 1 million daily transactions, **Hashed File Organization** is the best choice for fast point searches, while **Sorted File** is better when range queries are important.